CONTENT

CONTENT

S.NO.	DESCRIPTION	PAGE. NO.
1.	**INTRODUCTION** 1.1 SYSTEM SPECIFICATION 1.1.1 HARDWARE REQUIREMENTS 1.1.2 SOFTWARE REQUIREMENTS	1
2.	**SYSTEM STUDY** 2.1 EXISTING SYSTEM 2.1.1 DISADVANTAGES OF EXISTING SYSTEM 2.2 PROPOSED SYSTEM 2.2.1 ADVANTAGES OF PROPOSED SYSTEM 2.2.2 FEATURES OF PROPOSED SYSTEM	3
3.	**SYSTEM DESIGN AND DEVELOPMENT** 3.1 FILE DESIGN 3.2 INPUT DESIGN 3.3 CODE DESIGN 3.4 OUTPUT DESIGN 3.5 DATABASE DESIGN 3.6 SYSTEM DEVELOPMENT 3.6.1 MODULES 3.6.2 MODULE DESCRIPTION	5
4.	**TESTING AND IMPLEMENTATION** 4.1 IMPLEMENTATION 4.1.1 IMPLEMENTATION OF PASSWORD HASHING 4.1.2 IMPLEMENTATION OF LEAK DETECTION 4.1.3 IMPLEMENTATION OF PASSWORD STRENGTH 4.1.4 IMPLEMENTATION OF SECURITY SCORE 4.1.5 IMPLEMENTATIONOF PASSWORD GENERATOR 4.2 TESTING 4.2.1 TEST CLASS	10

	4.2.2 UNIT TESTING 4.2.3 INTEGRATION TESTING 4.2.4 SYSTEM TESTING 4.2.5 TESTING RESULT	
5.	**CONCLUSION**	14
6.	**BIBILIOGRAPHY**	15
7.	**APPENDICES** A. DATA FLOW DIAGRAM B. TABLE STRUCTURE C. SAMPLE CODING D. SAMPLE INPUT E. SAMPLE OUTPUT	16

ABSTRACT

ABSTRACT

The Password Leak Detection System is a security-based application developed to identify whether a user's password has been exposed in known leaked-password data. Password leaks are a major cybersecurity risk because compromised passwords can be misused to gain unauthorized access to user accounts.The proposed system helps users identify such risks and improve their password security.

In this system, the password entered by the user is converted into a secure SHA-256 hash and compared with stored leaked-password hash information. The system displays whether the password is detected as leaked or safe. It also analyzes password strength based on factors such as length, uppercase and lowercase letters, numbers, and special characters.

The system provides a Security Score to indicate the overall strength of the password and can generate strong passwords for users. A simple and user-friendly interface allows users to check their password security easily. By detecting leaked and weak passwords, the system helps users take preventive measures and encourages the use of strong and secure passwords. Thus, the Password Leak Detection System provides a simple and effective approach to improving password security and reducing the risk of unauthorized access.

The system also helps users understand the importance of using unique and strong passwords for different accounts. It provides useful security recommendations when a password is found to be weak or compromised. The password strength analysis feature encourages users to improve their passwords by adding suitable characters and increasing password length.

The Password Leak Detection System is designed to provide a simple, reliable, and user-friendly solution for password security. By combining password leak detection, hashing, strength analysis, and security score generation, the system helps users identify potential password risks. It promotes better password management practices and supports the protection of personal information from unauthorized access.

INTRODUCTION

CHAPTER – 1

INTRODUCTION

1.1 INTRODUCTION TO PASSWORD LEAK DETECTION SYSTEM

In today's digital world, passwords are widely used to protect personal information, online accounts, applications, and websites. Users use passwords for email, banking, social media, educational portals, and many other online services. However, weak, reused, or leaked passwords can create serious security risks. When a password is exposed, attackers may misuse it to gain unauthorized access to user accounts. Therefore, maintaining strong and secure passwords is an important part of cybersecurity.

The Password Leak Detection System is developed to identify whether a user's password has been exposed in known leaked-password information. The main purpose of this system is to help users understand the security condition of their passwords and take appropriate preventive measures. The system converts the entered password into a SHA-256 hash and compares it with stored leaked-password hash information. This approach helps in checking password exposure without storing the actual password in plain text.

The system also analyzes the strength of the entered password based on factors such as password length, uppercase letters, lowercase letters, numbers, and special characters. Based on these factors, the system provides a password strength result and a Security Score. This allows users to understand whether their password requires improvement and encourages them to create stronger passwords.

The system also includes a Strong Password Generator that helps users create secure passwords. Security recommendations are provided when a password is weak or detected as compromised. The application is designed with a simple and user-friendly interface so that users can easily enter a password and view its security status.

1.2 SYSTEM SPECIFICATION

The System Specification describes the hardware and software requirements needed to develop and execute the Password Leak Detection System. The system is designed to run on a standard computer or laptop and uses Python for implementing password processing, hashing, leak detection, and security analysis. The following specifications provide a suitable environment for developing and testing the proposed system.

1.2.1 HARDWARE REQUIREMENTS

Processor : Intel Core i3 or above

RAM : 4 GB or above

Hard Disk : 128 GB or above

Monitor : Standard display

Keyboard : Standard keyboard

Mouse : Standard mouse

1.2.2 SOFTWARE REQUIREMENTS

Operating System : Windows 10 / Windows 11

Programming Language : Python

Development Environment : Visual Studio Code

Frontend : HTML and CSS

Hashing Technique : SHA-256

Web Browser : Google Chrome / Microsoft Edge

SYSTEM STUDY

CHAPTER – 2

SYSTEM STUDY

2.1 EXISTING SYSTEM

The existing password security systems mainly focus on basic password authentication and password strength checking. They check password characteristics such as length, uppercase and lowercase letters, numbers, and special characters. However, these systems may not provide complete protection against leaked passwords or identify whether a password has been exposed in a data breach. They also provide limited support for password reuse detection and overall password security analysis. Therefore, users may continue using weak or compromised passwords without being aware of the associated security risks.

2.1.1 DISADVANTAGES OF EXISTING SYSTEM

- Basic systems mainly focus on password authentication and simple strength checking.
- They may not detect whether a password has been exposed in a data breach.
- Password reuse detection is generally not available.
- They provide limited information about the overall security condition of a password.
- Users may not receive proper warnings about compromised passwords.
- Strong password generation may not be available.
- They provide limited protection against password-related security risks.

2.2 PROPOSED SYSTEM

The proposed Password Leak Detection System is designed to provide a simple and effective method for improving password security. The system checks whether the password entered by the user matches any known leaked-password information stored in the system. The entered password is converted into a SHA-256 hash and compared with the stored leaked-password hashes without storing the actual password in plain text.

The system also analyzes password strength based on length, uppercase and lowercase letters, numbers, and special characters. It provides a Security Score to indicate the overall security level of the password. A Strong Password Generator is also provided to help users create secure passwords. The system uses a simple and user-friendly interface to display the password security result and recommendations.

By combining password leak detection, secure hashing, password strength analysis, security scoring, and strong password generation, the proposed system helps users identify password-related risks and improve their password security.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Uses SHA-256 hashing to process passwords securely.
- Analyzes password strength using different security conditions.
- Provides a Security Score for better understanding of password security.
- Helps users identify weak and potentially compromised passwords.
- Provides a Strong Password Generator for creating secure passwords.
- Does not require storing passwords in plain text.
- Provides a simple and user-friendly interface.
- Helps users improve their password management practices.

2.2.2FEATURES OF PROPOSED SYSTEM

1. Password Leak Detection – Checks whether the password is present in known leaked password data.

2. SHA-256 Hashing – Converts the password into a secure hash for comparison.

3. Password Strength Analysis – Checks length, uppercase, lowercase, numbers, special characters.

4. Security Score – Provides a score to indicate the password security level.

5. Strong Password Generator – Helps users create strong and secure passwords.

SYSTEM DESIGN

AND

DEVELOPMENT

CHAPTER – 3

SYSTEM DESIGN AND DEVELOPMENT

The Password Leak Detection System is designed to check password security in a simple and secure way. The system uses HTML and CSS for the user interface and Python for processing the password.

The entered password is converted into a SHA-256 hash and compared with stored leaked-password hash information. The system also checks password strength based on length, uppercase and lowercase letters, numbers, and special characters.

Finally, the system displays the leak status, password strength, and Security Score. The system is divided into separate files such as app.py, password_security.py, leaked_passwords.py, index.html, and style.css, making it easy to develop and maintain

3.1 FILE DESIGN

→ app.py → Main application

→ password_security.py → Password hashing and strength checking

→ leaked_passwords.py → Stores leaked password hash data

→ index.html → User interface

→ style.css → Webpage design and styling

→ All files → Work together to run the Password Leak Detection System

3.2 INPUT DESIGN

→ User opens the Password Leak Detection System.

→ User enters a password in the input field.

→ The entered password is taken as the system input.

→ The password is converted into SHA-256 hash.

→ The generated hash is compared with stored leaked-password hashes.

→ The system processes the input and generates the security result.

3.3 CODE DESIGN

→ Python is used for developing the main application.

→ app.py contains the main application code.

→ password_security.py contains password security functions.

→ leaked_passwords.py contains leaked password hash data.

→ HTML is used to create the user interface.

→ CSS is used to design the webpage.

→ SHA-256 is used for password hashing.

→ All codes work together to provide password leak detection

.

3.4 OUTPUT DESIGN

→ System displays the password leak status.

→ It shows whether the password is leaked or safe.

→ Password strength is displayed.

→ Security Score is displayed.

→ Security recommendations are provided.

→ Strong password can be generated for the user.

→ The final security result is displayed clearly on the webpage.

3.5 DATABASE DESIGN

→ The Password Leak Detection System does not use a separate database.

→ Leaked password hash information is stored in the Python file.

→ The system compares the entered password hash with the stored hash data.

→ No plain-text password is stored in the system.

→ This simple design makes the project easy to develop and maintain.

3.6 SYSTEM DEVELOPMENT

The Password Leak Detection System is developed using Python, HTML, and CSS. The system accepts a password from the user and converts it into a SHA-256 hash for secure processing. The generated hash is compared with the stored leaked-password hash information to identify whether the password is compromised. The system also analyzes the password strength based on length, uppercase and lowercase letters, numbers, and special characters. Based on the analysis, the system displays the leak status, password strength, and Security Score. A simple and user-friendly interface is provided to make the system easy to use. The developed system helps users identify password security risks and improve their password protection.

During development, different types of passwords are tested to verify the working of the system. The system checks both strong and weak passwords and displays the appropriate security result. The testing process helps to identify errors and ensure that the system produces correct and reliable results.

The final system provides a simple interface through which users can check their password security easily. The modular file structure also makes the system easier to understand, maintain, and modify. Thus, the developed system provides a basic and effective solution for detecting password leaks and improving password security.

3.6.1 MODULES

→ **Password Input Module** → Accepts the password entered by the user.

→ **Password Hashing Module** → Converts the password into a SHA-256 hash.

→ **Password Strength Module** → Checks the strength of the entered password.

→ **Security Score Module** → Calculates and displays the password security score.

→ **Password Generator Module** → Generates strong passwords for the user.

→ **Result Display Module** → Displays the final security status and recommendations.

3.6.2 MODULE DESCRIPTION

1. Password Input Module

The Password Input Module is the first component of the system. It accepts the password entered by the user through the application interface. The module validates the input and sends it to the next process for security analysis. A simple and user-friendly interface is provided to make password checking easier.

2. Password Hashing Module

The Password Hashing Module converts the entered password into a SHA-256 hash value. Hashing improves security because the actual password is not stored or processed directly. The generated hash is used for password comparison and leak detection.

3. Leak Detection Module

This module compares the generated password hash with the available leaked-password hash information. If a matching hash is found, the system identifies the password as leaked. If no match is found, the password is considered safe. This module helps users avoid using compromised passwords.

4. Password Strength Module

The Password Strength Module analyzes the entered password using different conditions. It checks password length, uppercase letters, lowercase letters, numbers, and special characters. Based on these factors, the system identifies whether the password is weak, medium, or strong.

5. Security Score Module

This module calculates a Security Score for the entered password. The score is generated based on password characteristics and strength. A higher score indicates better password security. The Security Score helps users understand the overall protection level of their passwords.

6. Password Generator Module

The Password Generator Module creates strong passwords automatically. The generated passwords include uppercase letters, lowercase letters, numbers, and special characters. This module helps users create secure passwords and improve account safety.

7. Result Display Module

The Result Display Module shows the final results to the user. It displays the leak status, password strength, Security Score, and recommendations. The information is presented clearly so that users can easily understand the security condition of their passwords. Overall, the modules work together to provide an effective Password Leak Detection System. Each module performs a specific function, and the integration of all modules improves password security and user awareness.

TESTING

AND

IMPLEMENTATION

CHAPTER – 4

TESTING AND IMPLEMENTATION

4.1 IMPLEMENTATION

The Password Leak Detection System is implemented using Python, HTML, and CSS. Python is used for application logic, password hashing, leak detection, password strength analysis, and security score calculation. HTML to create the user interface, while CSS is used for webpage design. SHA-256 hashing is used to securely process the password.

4.1.1 IMPLEMENTATION OF PASSWORD HASHING

→ User enters a password.

→ The password is converted into a SHA-256 hash.

→ The generated hash is used for comparison.

→ The actual password is not stored as plain text.

4.1.2 IMPLEMENTATION OF LEAK DETECTION

→ The generated password hash is compared with stored leaked-password hashes.

→ If a match is found, the password is identified as leaked.

→ If no match is found, the password is displayed as not leaked.

4.1.3 IMPLEMENTATION OF PASSWORD STRENGTH

→ Password length is checked.

→ Uppercase and lowercase letters are checked.

→ Numbers and special characters are checked.

4.1.4 IMPLEMENTATION OF SECURITY SCORE

→ The password characteristics are analyzed.

→ A Security Score is calculated.

→ The score is displayed to the user along with the security level.

4.1.5 IMPLEMENTATION OF PASSWORD GENERATOR

→ The system generates a random password.

→ Different characters are included in the generated password.

→ The generated password can be used as a strong password.

4.2 TESTING

Testing is performed to verify that the Password Leak Detection System works correctly. Different password inputs are given to the system and the generated results are checked.

4.2.1 TEST CASES

Test case	Input	Expected Result
1	Leaked password	Leaked password displayed
2	Safe password	Not leaked status displayed
3	Weak password	Weak strength displayed
4	Strong password	Strong strength displayed
5	Password with mixed characters	Security score displayed
6	Generative password	Strong password displayed

4.2.2 UNIT TESTING

Each module is tested separately to verify its individual functionality. Password hashing, leak detection, strength analysis, Security Score, and password generation are checked independently.

4.2.3 INTEGRATION TESTING

All modules are combined and tested together. The testing verifies that the user input, password processing, leak detection, strength analysis, and result display work correctly as a complete system.

4.2.4 SYSTEM TESTING

The complete Password Leak Detection System is tested to ensure that all functions work correctly. Different types of passwords are entered and the final results are verified.

4.2.5 TESTING RESULT

The testing of the Password Leak Detection System was carried out using different types of password inputs to verify the functionality and reliability of the application. The system was tested with weak passwords, strong passwords, leaked passwords, and passwords containing different combinations of characters.

The testing confirmed that the system correctly converts the entered password into a SHA-256 hash and compares the generated hash with the stored leaked-password hash information. When a matching hash is found, the system displays the password as leaked. When no matching hash is found, the system displays the password as not leaked.

The password strength checking function was also tested with different password combinations. The system successfully identifies passwords based on their length and character combinations. Weak and strong passwords are classified accordingly, helping users understand the security level of their passwords.

The Security Score and security recommendations were also verified during testing. The system provides suitable results based on the password characteristics. The Password Generator module was tested to ensure that it can generate strong passwords containing different types of characters.

Overall, the testing results show that the major functions of the Password Leak Detection System work as expected. The system successfully performs password hashing, leak detection, password strength analysis, security scoring, and password generation. The application provides clear and understandable results to the user and meets the basic objectives of the proposed system

CONCLUSION

CHAPTER – 5

CONCLUSION

The Password Leak Detection System is developed to provide a simple and effective solution for identifying password-related security risks. In today's digital environment, passwords are widely used to protect personal accounts, applications, websites, and sensitive information. Weak, reused, or leaked passwords can increase the possibility of unauthorized access. Therefore, checking password security and encouraging users to create strong passwords are important for protecting digital information.

The developed system provides a simple method to check whether a password is present in the available leaked-password information. The password entered by the user is converted into a SHA-256 hash before comparison. This helps the system process the password securely without storing the actual password in plain text. The generated hash is compared with the stored leaked-password hash information, and the system displays the appropriate leak status to the user.

The system also includes password strength analysis to evaluate the security of the entered password. It checks important characteristics such as password length, uppercase letters, lowercase letters, numbers, and special characters. Based on these conditions, the system provides a password strength result and calculates a Security Score. This helps users understand the security level of their passwords and identify areas that require improvement.

A Strong Password Generator is also included in the system to help users create secure passwords. The generated passwords can contain different types of characters, making them more suitable for protecting user accounts. The system provides a simple and user-friendly interface so that users can easily enter a password, check its security status, view the strength result, and understand the recommendations provided by the application.

Overall, the Password Leak Detection System provides a basic and useful approach to improving password security. It helps users identify potentially leaked and weak passwords and encourages them to use stronger passwords. The system can also be enhanced in the future by adding more advanced security features and a larger collection of leaked-password hash data.

BIBILIOGRAPHY

CHAPTER – 6

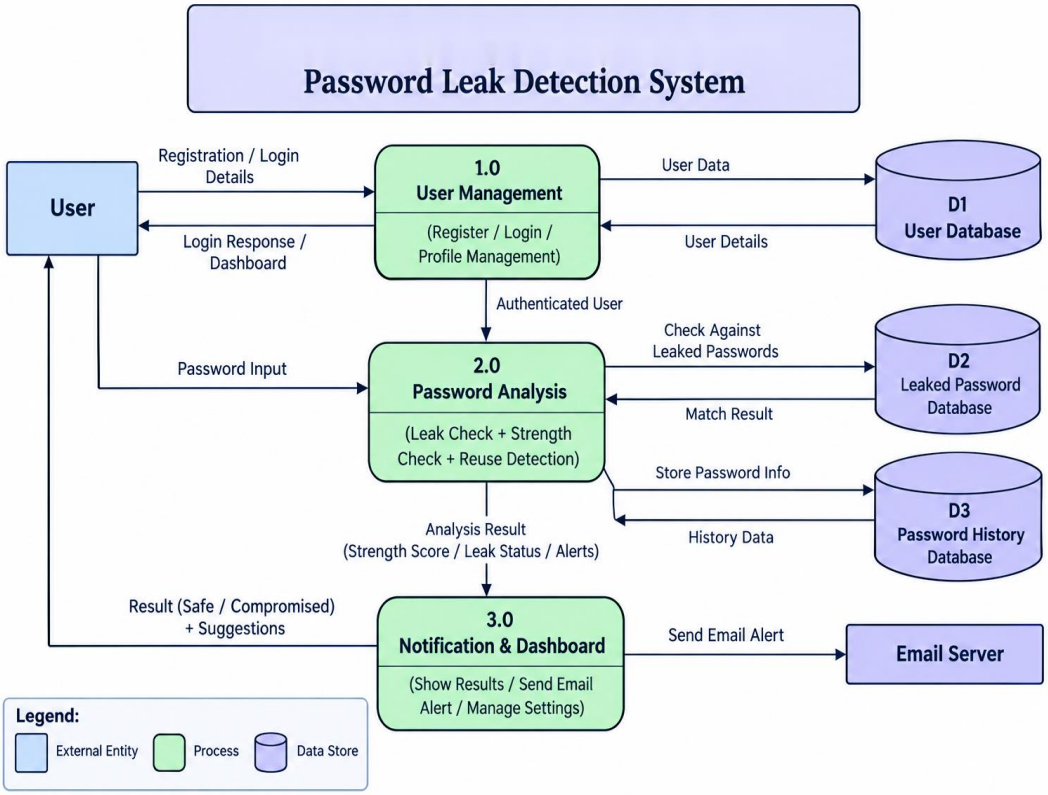
BIBILIOGRAPHY

1. William Stallings, Cryptography and Network Security: Principles and Practice, Pearson Education.
2. Behrouz A. Forouzan, Cryptography and Network Security, McGraw-Hill Education.
3. Charles P. Pfleeger and Shari Lawrence Pfleeger, Security in Computing, Prentice Hall.
4. Michael E. Whitman and Herbert J. Mattord, Principles of Information Security, Cengage Learning.
5. OWASP Foundation, OWASP Password Storage Cheat Sheet.
6. OWASP Foundation, OWASP Authentication Cheat Sheet.
7. National Institute of Standards and Technology (NIST), Digital Identity Guidelines.
8. National Institute of Standards and Technology (NIST), Authentication and Lifecycle Management.
9. Python Software Foundation, Python Documentation.
10. Python Software Foundation, hashlib – Secure Hashes and Message Digests.
11. Python Software Foundation, getpass – Portable Password Input.
12. Mozilla Developer Network, HTML Documentation and Web Development Resources.
13. Mozilla Developer Network, HTML Forms and Input Elements.
14. W3Schools, HTML Tutorial and Web Development Reference.
15. W3Schools, Python Programming Tutorial and Examples.
16. GeeksforGeeks, Password Security and Password Hashing Techniques.
17. GeeksforGeeks, Cryptography and Hashing in Python.
18. SANS Institute, Password Security and Authentication Best Practices.
19. Cloudflare, Data Breaches and Password Security Resources.
20. Google Safety Center, Online Security and Password Protection Guidelines.

APPENDICES

APPENDICES

A. DATA FLOW DIAGRAM



B. TABLE STRUCTURE

FIELD NAME	DATA TYPE	DESCRIPTION
Password	String	Stores the password entered by the user.
Password Hash	String	Stores the SHA-256 hash of the password.
Leak Status	String	Shows whether the password is leaked or not leaked.
Password Strength	String	Shows Weak, Medium, or Strong.
Security Score	String	Shows the calculated security score.
Suggestions	String	Provides security recommendations.
Generated Password	String	Contains the strong password generated by the system

C. SAMPLE CODING

```
import hashlib
import getpass

# Sample leaked password hashes
LEAKED_PASSWORDS = {
    hashlib.sha256("password123".encode()).hexdigest(),
    hashlib.sha256("12345678".encode()).hexdigest(),
    hashlib.sha256("qwerty123".encode()).hexdigest()
}

def check_password_strength(password):
    score = 0

    if len(password) >= 8:
        score += 20

    if any(c.isupper() for c in password):
        score += 20

    if any(c.islower() for c in password):
        score += 20

    if any(c.isdigit() for c in password):
        score += 20

    if any(c in "!@#$%^&*" for c in password):
        score += 20

    return score
```

```
def check_password_leak(password):
    password_hash = hashlib.sha256(password.encode()).hexdigest()

    if password_hash in LEAKED_PASSWORDS:
        return "COMPROMISED"
    else:
        return "SAFE"



print("=====================================")
print("   PASSWORD LEAK DETECTION SYSTEM")
print("=====================================")

username = input("Enter your username: ")
password = getpass.getpass("Enter your password: ")

leak_status = check_password_leak(password)
security_score = check_password_strength(password)

print("\n---------- RESULT ----------")
print("Username       :", username)
print("Leak Status    :", leak_status)
print("Security Score  :", security_score, "/ 100")

if security_score >= 80:
    print("Password Strength: STRONG")
elif security_score >= 50:
    print("Password Strength: MEDIUM")
```

```
else:
        print("Password Strength: WEAK")

    if leak_status == "COMPROMISED":
        print("WARNING: This password was found in leaked data.")
        print("Suggestion: Please change your password.")
    else:
        print("Your password was not found in the sample leaked database.")

    print("------------------------------")
```

D. SAMPLE INPUT

```
    ======================================
      PASSWORD LEAK DETECTION SYSTEM
    ======================================

    Enter your username: projectuser01
    Enter your password: Password@12345
```

E. SAMPLE OUTPUT

```
==========================================
  PASSWORD LEAK DETECTION SYSTEM
==========================================

---------- RESULT ----------
Username        : projectuser01
Leak Status     : SAFE
Security Score  : 100 / 100
Password Strength: STRONG

Your password was not found in the sample leaked database.
------------------------------
```